

Quiz

Assume you are connecting to a web server using HTTP 1.1 w/ persistent connection.

Calculate the # of RTTs needed to fetch and receive an index.html with 5 embedded jpeg images. Assume each image can fit in one packet.

Answer: 2 RTTs are needed for fetching index.html, and then 1 RTT for each embedded image. Thus, the total duration is $2 + 5 = 7$ RTTs.

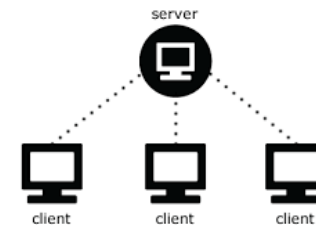
Application layer: overview

- Principles of network applications
- Web and HTTP
- Video streaming and content distribution networks
- Socket programming with UDP and TCP
- The Domain Name System DNS
- **P2P applications**

Recap: Two Communication Architecture

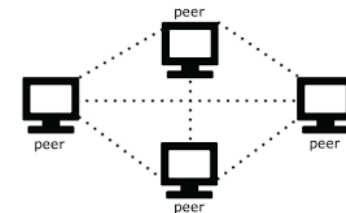
■ Client-server

- Server is *always on* with a fixed address, services many clients, do not initiate requests with clients
- Client is *sometimes on*, sends requests to the server



■ Peer-to-peer

- No always-on server at all
- Hosts can come and go, change addresses, can initiate communication with any others



P2P Network #1: Napster



- Sharing audio (typically MP3 music) files
 - Company founded in May 1999
 - First lawsuit in Sept. 1999
 - 80 million users in 2000
 - Ran out of business due to lawsuits in mid 2001
 - Online music store, merged with Rhapsody from BestBuy in 2011
- Directory service
 - Napster server maintains the list of music files each client has
 - When a client searches on music title or performer, server returns IP addresses of online clients with the file
 - Client requests files directly from other online clients

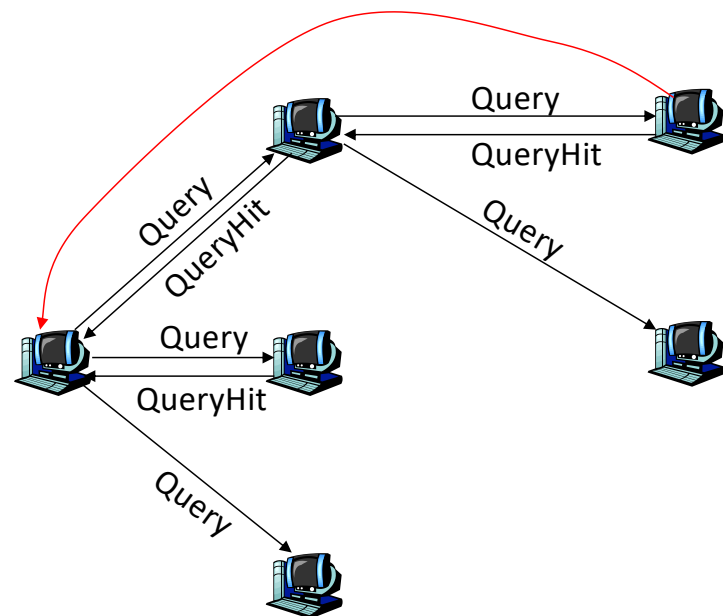
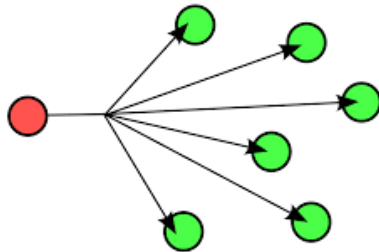
Napster Technology

- File transfer is decentralized, yet locating content is highly centralized
 - Server keeps updating the directory
 - Peer-to-peer file transfer, no load on the server
- No security
 - Clear-text passwords and other vulnerability
- Rank clients by bandwidth and response time
- Limitations?
 - Single point of failure ☹️



P2P Network #2: Gnutella

- Released in 2000
- No central server at all
- Query by flooding your requests to your neighbors



Gnutella: Pros and Cons

■ Pros

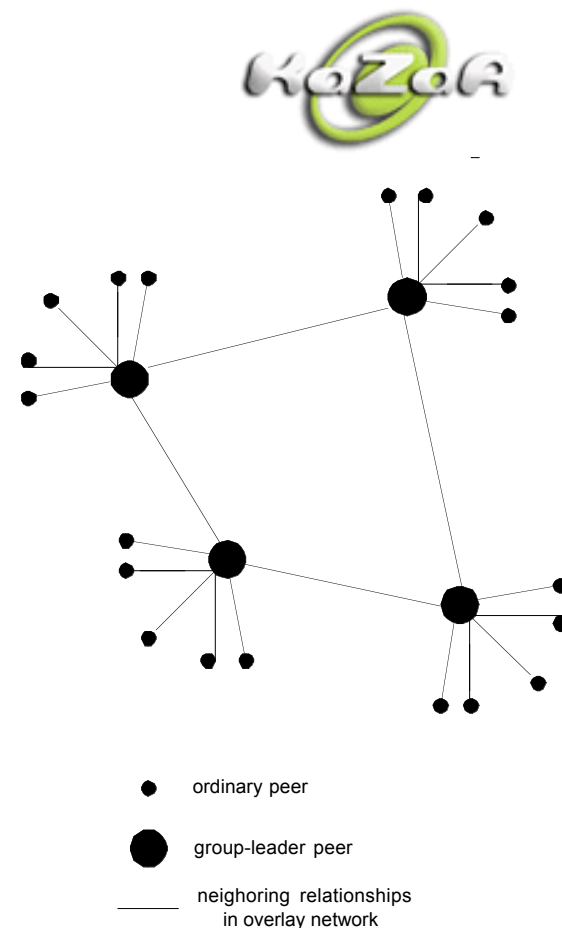
- Fully decentralized, distribute search cost
- Processing per node allows powerful search semantics

■ Cons

- Search scope can get huge
- Search time can get quite long
- High overhead as node come and go often

P2P Network #3: KaZaA

- Created by Dutch company in 2001
- Smart query flooding
 - Flood query within a small set of nodes (super nodes)
 - Heterogeneous nodes: peer and super nodes
- Combination of Napster and Gnutella
- Super node tracks the content of all its children
- Why super node?
 - **Query consolidation:** propagating query to sub-node can take longer than the super-node
 - **Stability:** nodes with high up-time as super nodes



Finally, BitTorrent

- Key motivation: popular content
 - Popularity exhibits temporal locality (flash crowds)
- Focus on efficient **fetching**, not searching
 - Distribute the same file to many peers
- Prevent free-loading
- Components

Seed: peer with the entire file



Peer/Leecher: peer with incomplete copy of the file



Tracker: allow peers to find each other

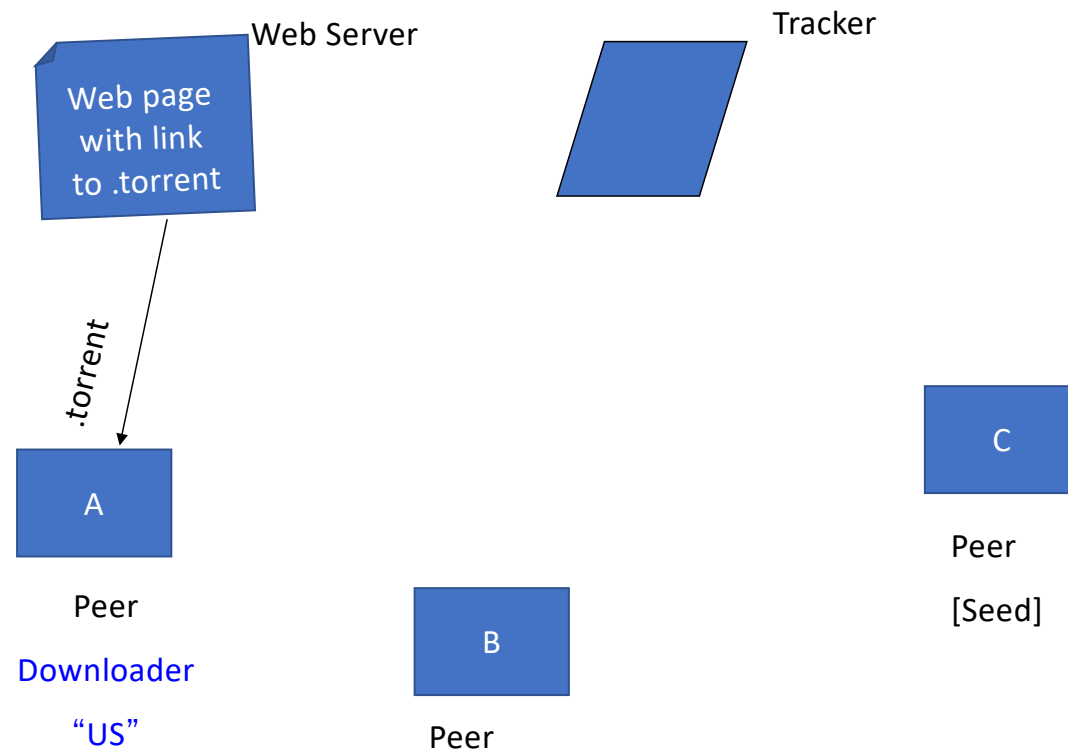


Application Layer: 2-9

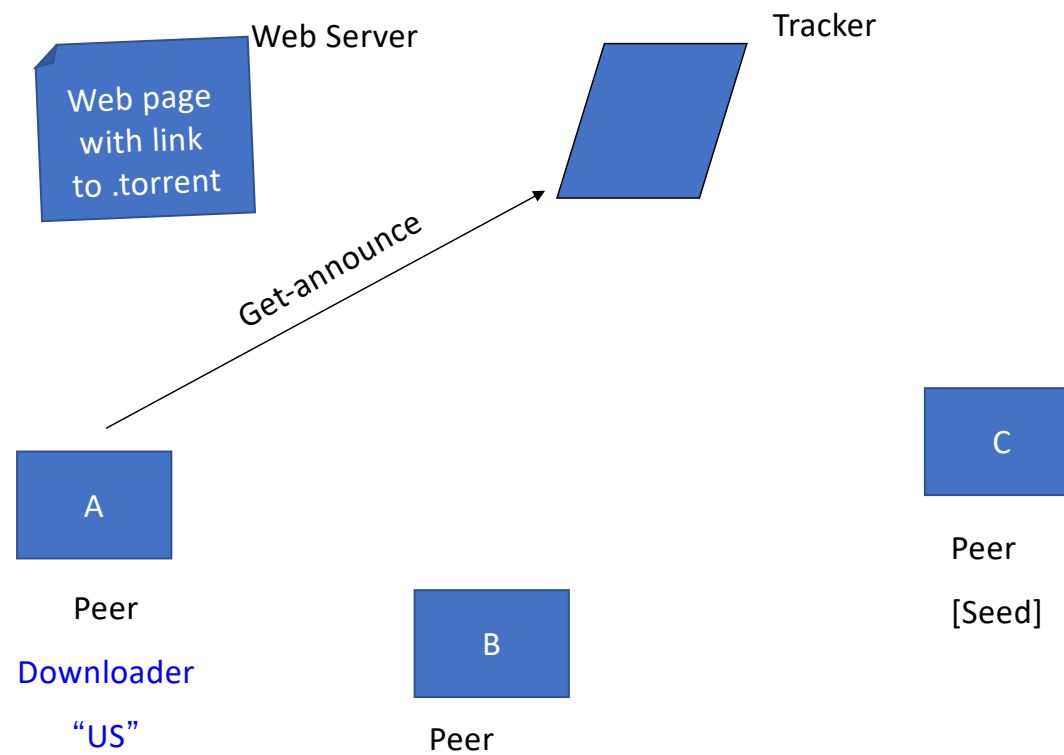
Property #1: Simultaneous Downloading

- Divide a large file into many pieces (chunks)
- Replicate different pieces on different peers
- A peer can upload and download pieces at the same time
- *A peer can retrieve **different pieces** of the file from **different peers at the same time***

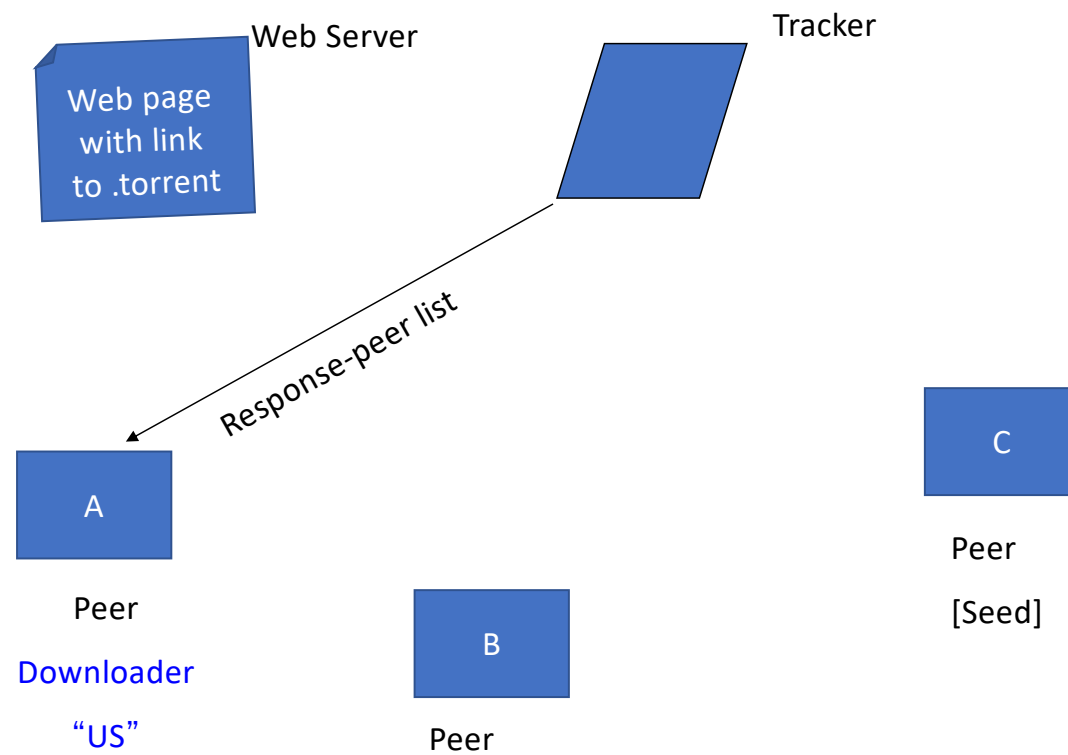
Example



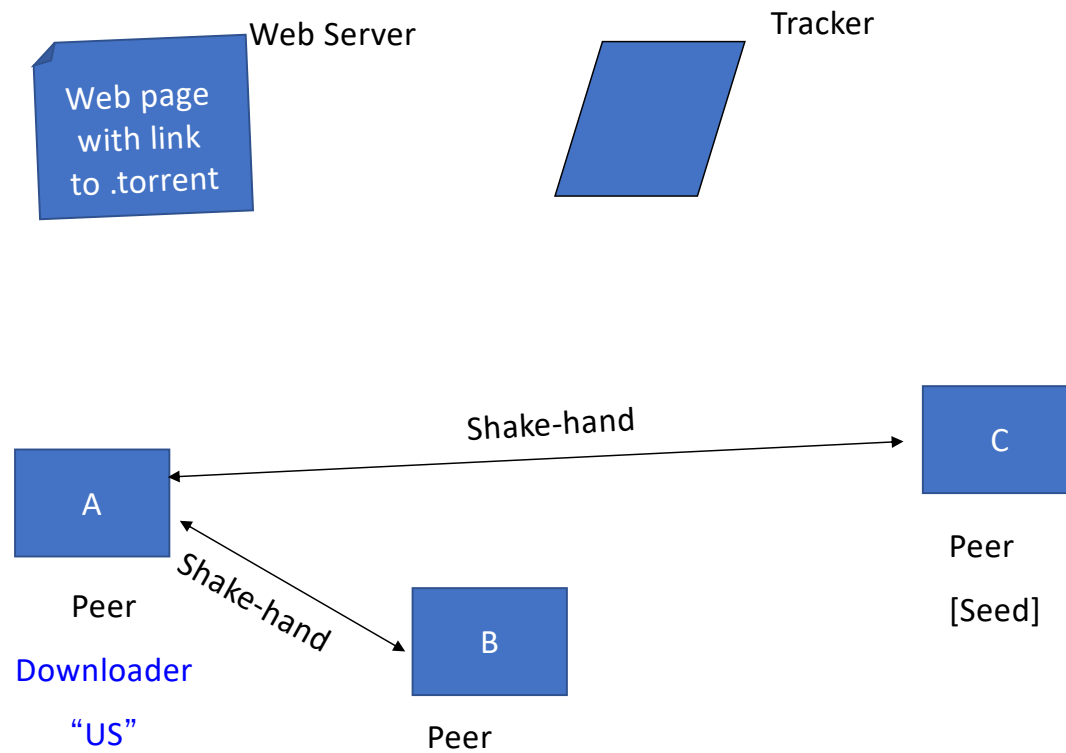
Example



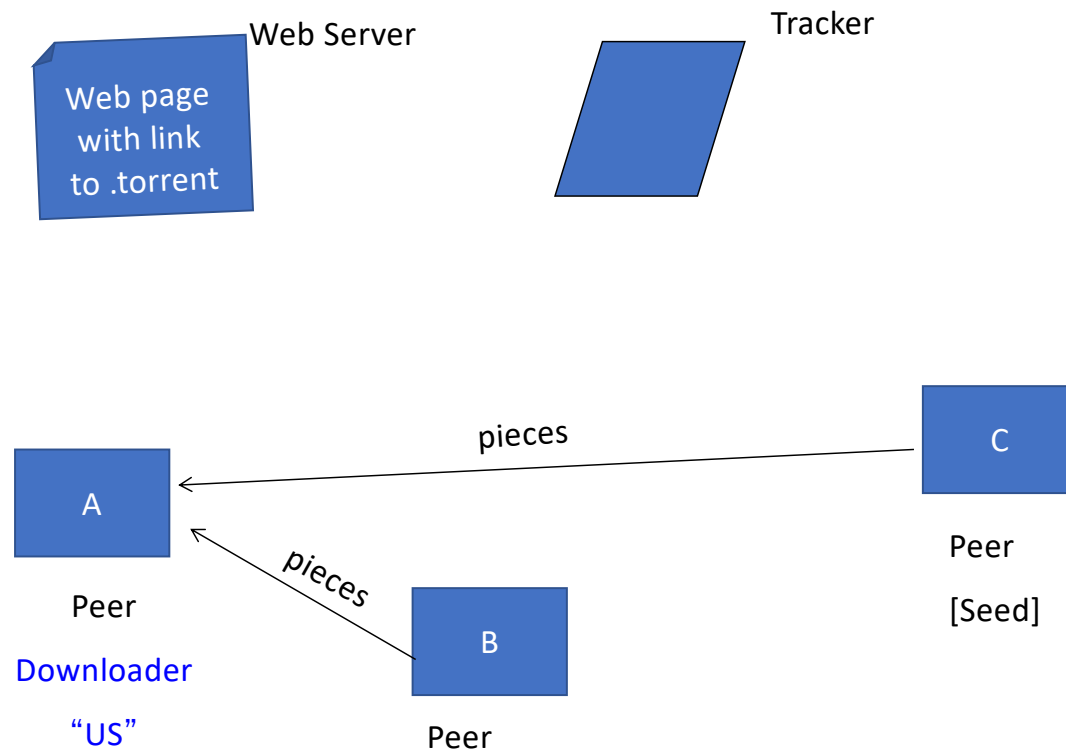
Example



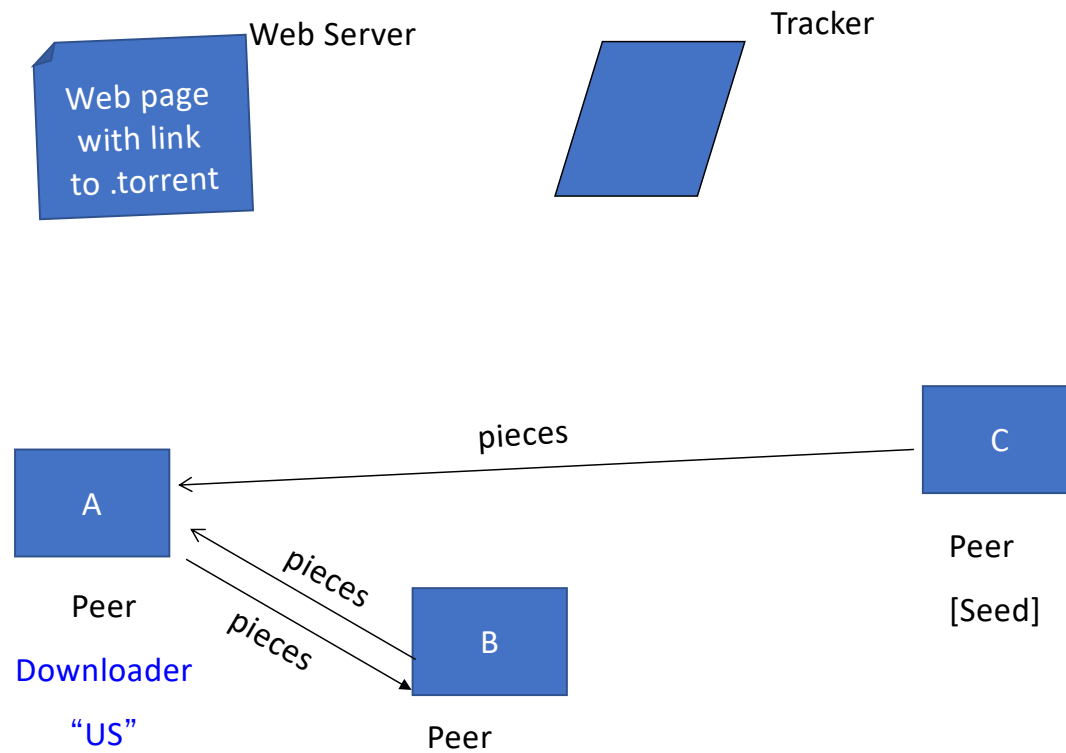
Example



Example



Example



Property #2: Anti Free-Riding

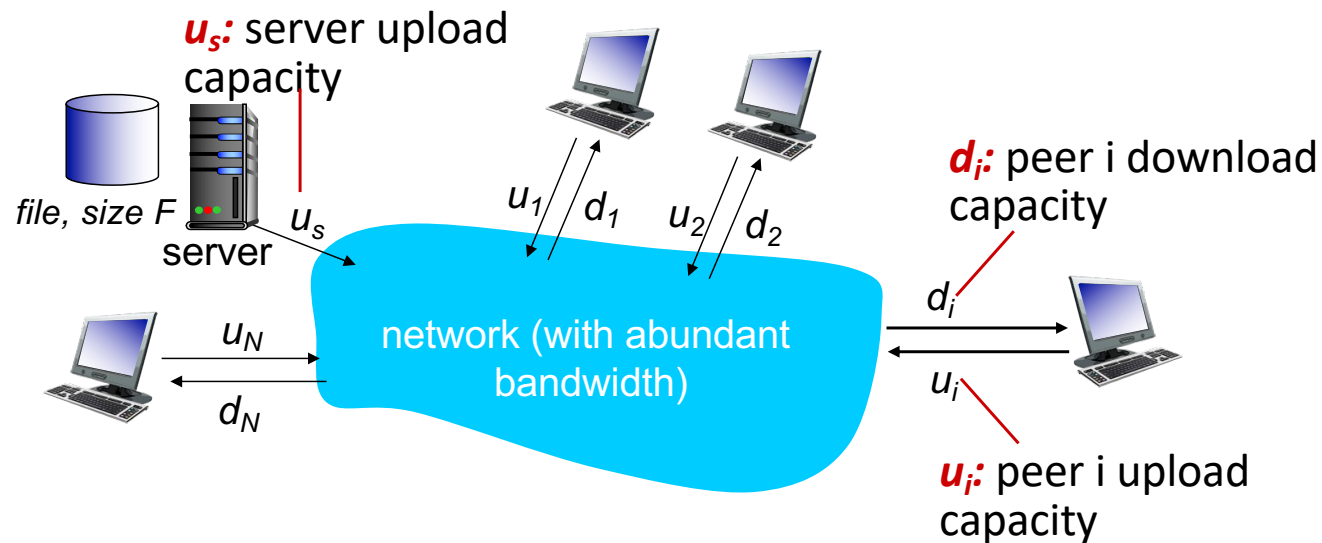
- Vast majority of users are free-riders
 - Most share no files and answer no queries
 - Other limit the # of connections or upload speed
- A few peers essentially act as servers
- Tit-for-Tat
 - Peers decide which peers they upload files to at the high rate
 - Choke other peers, i.e., do not send them chunks
 - Re-evaluate top 4 peers every 10 seconds
 - Occasionally let some free riders download (opportunistic unchoking)
 - newly chosen peer may join top 4



File distribution: client-server vs P2P

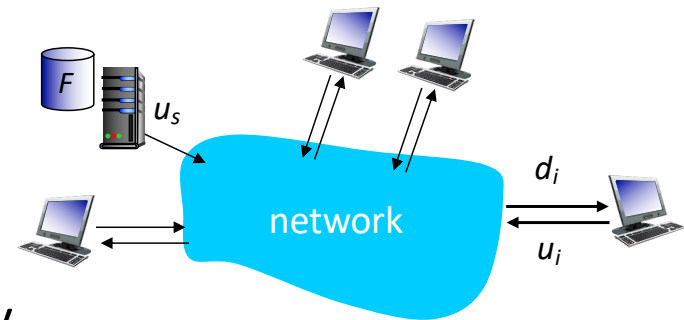
Q: how much time to distribute file (size F) from one server to N peers?

- peer upload/download capacity is limited resource



File distribution time: client-server

- **server transmission:** must sequentially send (upload) N file copies:
 - time to send one copy: F/u_s
 - time to send N copies: NF/u_s
- **client:** each client must download file copy
 - d_{min} = min client download rate
 - min client download time: F/d_{min}



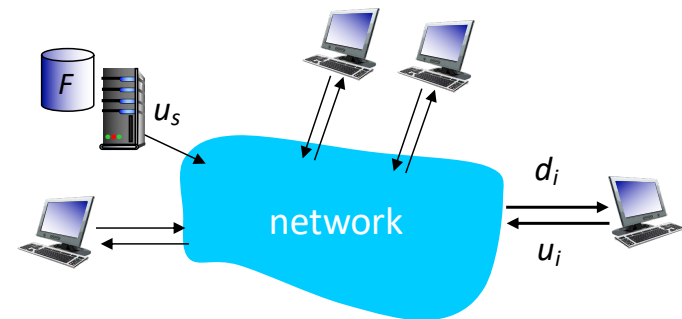
time to distribute F
to N clients using
client-server approach

$$D_{c-s} \geq \max\{NF/u_s, F/d_{min}\}$$

increases linearly in N

File distribution time: P2P

- **server transmission:** must upload at least one copy:
 - time to send one copy: F/u_s
- **client:** each client must download file copy
 - min client download time: F/d_{min}
- **clients:** as aggregate must download NF bits
 - max upload rate (limiting max download rate) is $u_s + \sum u_i$



time to distribute F
to N clients using
P2P approach

$$D_{P2P} \geq \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$$

increases linearly in N ...
... but so does this, as each peer brings service capacity

Client-server vs. P2P: example

client upload rate = u , $F/u = 1$ hour, $u_s = 10u$, $d_{min} \geq u_s$

